SPECFIX – AI-Driven Requirement

Ambiguity Detection and Test Specification System

A Phase 1 (60%) Progress Report submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering
Specialization in
Artificial Intelligence and Machine Learning

By

Shrikant Sharma (2415500449)
Raghav Bansal (2415500377)
Udit Bhatt (2415500484)

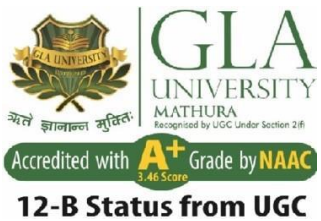
Group No. 138

Under the Guidance of

Mr. Preshit Mangesh Desai

Department of Computer Engineering & Applications

Institute of Engineering & Technology



GLA University
Mathura- 281406, INDIA

October, 2026

DECLARATION

We hereby declare that the work which is being presented in the B.Tech. Project **"SpecFix – AI-Driven Requirement Ambiguity Detection and Test Specification System"**, in partial fulfilment of the requirements for the award of the Bachelor of Technology in Computer Science and Engineering Specialization in Artificial Intelligence and Machine Learning and submitted to the Department of Computer Engineering and Applications of GLA University, Mathura, is an authentic record of our own work carried out under the supervision of Mr. Preshit Mangesh Desai, Technical Trainer.

The contents of this project report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree.

Sign:
Name of Candidate: Shrikant Sharma
Univ. Roll No.: 2415500449
Course: B.Tech. CSE (AIML)

Sign:
Name of Candidate: Raghav Bansal
Univ. Roll No.: 2415500377
Course: B.Tech. CSE (AIML)

Sign:
Name of Candidate: Udit Bhatt
Univ. Roll No.: 2415500484
Course: B.Tech. CSE (AIML)

CERTIFICATE

This is to certify that the above statements made by the candidates are correct to the best of my/our knowledge and belief.

Supervisor
(Mr. Preshit Mangesh Desai)
Technical Trainer
Dept. of Computer Engg. & App.

Project Co-ordinator
(Dr. Mayank Srivastava)
Associate Professor
Dept. of Computer Engg. & App.

Program Co-ordinator
(Dr. Nikhil Govil)
Associate Professor
Dept. of Computer Engg. & App.

Date: 21 October, 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported and guided us throughout this project. First and foremost, we extend our heartfelt thanks to our respected guide, Mr. Preshit Mangesh Desai (Technical Trainer), for his continuous encouragement, valuable feedback and insightful suggestions at every stage of the project titled **"SpecFix – AI-Driven Requirement Ambiguity Detection and Test Specification System"**. His guidance has been fundamental in shaping both the direction and the discipline of this work.

We are deeply thankful to the Department of Computer Engineering and Applications and to our institution for providing a supportive learning environment, access to essential resources and the facilities that enabled the smooth execution of this project. Building SpecFix has significantly strengthened our knowledge of large language models, database design, web engineering and software testing, as well as our teamwork and problem-solving skills.

We would also like to thank the Head of the Department, Mr. Ashok Bhansali, for his constant support and motivation. We are grateful to all faculty members of the department for their guidance, and to our classmates and friends for their cooperation and suggestions. Lastly, we owe our deepest appreciation to our families, who have been a continuous source of strength, patience and motivation.

Shrikant Sharma
2415500449

Raghav Bansal
2415500377

Udit Bhatt
2415500484

ABSTRACT

Software requirements are usually written as short tickets or user stories in tools such as Jira. These tickets are frequently ambiguous or incomplete: a word such as “quickly” hides a decision, an acceptance criterion cannot be tested, or an edge case is never mentioned. Developers fill the gaps with assumptions, and the mismatch with business intent is discovered late — in QA or after release — when it is most expensive to fix.

SpecFix is an AI-assisted requirements-quality platform that catches these gaps before development starts. A ticket goes in; structured *flags* come out, each with a category (missing information, vague language, contradiction, unhandled edge case, security/compliance gap or untestable criterion), the exact text it refers to, why it matters, and a question for the product manager. An *Extractor* language-model call proposes candidate flags and an *Adversarial Critic* call removes false alarms. A human reviewer then accepts, edits or dismisses every flag in a web dashboard, with an append-only audit trail and no automatic resolution path.

Once a ticket is approved, the *Spec Compiler* turns its acceptance criteria into Gherkin scenarios, and a deterministic generator turns each approved scenario into a Playwright test scaffold with marked placeholder selectors. A traceability graph backed by pgvector embeddings shows clause-level coverage and detects scenarios that drift when a ticket changes.

The system is a TypeScript monorepo (Next.js 15, Supabase PostgreSQL with Row-Level Security and pgvector, Azure OpenAI / OpenAI, zod, Vitest). Every model output is schema-validated, ticket text is always treated as data, every model call is logged with its cost, and the Critic’s precision is only reported next to a random-pruning control. On an 11-ticket gold set the Spec Compiler covered 91.7% of acceptance-criteria clauses, made one unsupported coverage claim in 42 scenarios (2.4%), and produced 42 of 42 valid scaffolds at about US$0.0004 per ticket.

TABLE OF CONTENTS

Declaration ii
Certificate iii
Acknowledgement iv
Abstract v
Table of Contents vi
List of Tables vii
List of Figures vii

CHAPTER 1: INTRODUCTION 1
1.1 Overview and Motivation 1
1.2 Objectives 2
1.3 Summary of Similar Applications 3
1.4 Organization of the Project Report 3

CHAPTER 2: SOFTWARE REQUIREMENT ANALYSIS 3
2.1 Introduction to Requirement Analysis 4
2.2 Feasibility Study 4
2.3 Functional Requirements 5
2.4 Non-Functional Requirements 5
2.5 Use Case Descriptions 6
2.6 Module Descriptions 6

CHAPTER 3: SOFTWARE DESIGN 7
3.1 System Architecture Overview 7
3.2 Data Flow Diagrams 7
3.3 UML Diagrams 9
3.4 Database Design 12

CHAPTER 4: IMPLEMENTATION AND USER INTERFACE 13
4.1 Technology Stack Used 13
4.2 AI Architecture 13
4.3 Frontend Implementation 15

CHAPTER 5: SOFTWARE TESTING 18
5.1 Testing Approach 18
5.2 Unit Testing 18
5.3 Test Case Specifications 18
5.4 Multi-Tenant Isolation Testing 19
5.5 Evaluation Results 19

CHAPTER 6: CONCLUSION 20
6.1 Current Status & Future Work 20

CHAPTER 7: SUMMARY 21
7.1 Summary 21

REFERENCES 22

APPENDIX A: ABBREVIATIONS 23

APPENDIX B: SAMPLE COMMANDS 23

LIST OF TABLES

Table 1.1	Comparison with Existing Tools and Approaches	3
Table 2.1	Feasibility Study Summary	4
Table 2.2	Functional Requirements of SpecFix	5
Table 2.3	Non-Functional Requirements of SpecFix	5
Table 2.4	Main Use Cases	6
Table 2.5	Modules of SpecFix	6
Table 3.1	Principal Database Tables	12
Table 4.1	Technology Stack Summary	13
Table 5.1	Unit Testing Summary by Package (offline run)	18
Table 5.2	Black Box and White Box Test Cases	18
Table 5.3	Generation Evaluation Results (gold-v1)	19
Table 6.1	Project Status by Milestone	20

LIST OF FIGURES

Figure 1.1	System Architecture of SpecFix	2
Figure 3.1	Level 0 DFD – Context Diagram	8
Figure 3.2	Level 1 DFD – Main Data Flow	8
Figure 3.3	Use Case Diagram of SpecFix	9
Figure 3.4	Class Diagram of the Core Domain	10
Figure 3.5	Sequence Diagram – Extractor and Critic Analysis	11
Figure 3.6	Entity Relationship Diagram	12
Figure 4.1	Two-Pass Analysis Pipeline and Human Review Gate	14
Figure 4.2	Spec Compiler Pipeline with Two Human Gates	15
Figure 4.3	Review Queue Page	16
Figure 4.4	Ticket Review Page with Flag Cards	16
Figure 4.5	Specs Tab: Scenario Review, Coverage and Scaffold	17

CHAPTER 1
INTRODUCTION

1.1 Overview and Motivation

Most software teams describe what they want built as tickets: a title, a short description and a few acceptance criteria. These tickets are written quickly and are rarely complete. A criterion such as “the refund should show up quickly” leaves the developer to decide what “quickly” means, and “the customer is notified” does not say how. Each missing decision is made silently by whoever writes the code, and the later such a defect is found, the more expensive it is to fix. Typical problems are vague words, missing values or rules, contradictions, unhandled edge cases, unstated security or audit needs, and criteria that no test could pass or fail.

Specialist requirements-quality tools exist, but they target regulated industries and sit outside the everyday ticket workflow. General chat assistants can “review a ticket”, but their answers are unstructured, unmeasured and easy to manipulate with text hidden inside the ticket. **SpecFix** fills this gap: a ticket goes in, structured flags come out, a human resolves each one, and the ticket is marked ready for development. The approved criteria are then turned into Gherkin scenarios and Playwright test scaffolds, with a traceability graph from requirement to test.

Why This Project?

- Every model response is schema-validated, and ticket text is treated as *data, never instructions*.
- A human resolves every flag and approves every generated scenario and scaffold.
- The Critic is tested against a random-pruning control; tenants are isolated and every model call is costed.

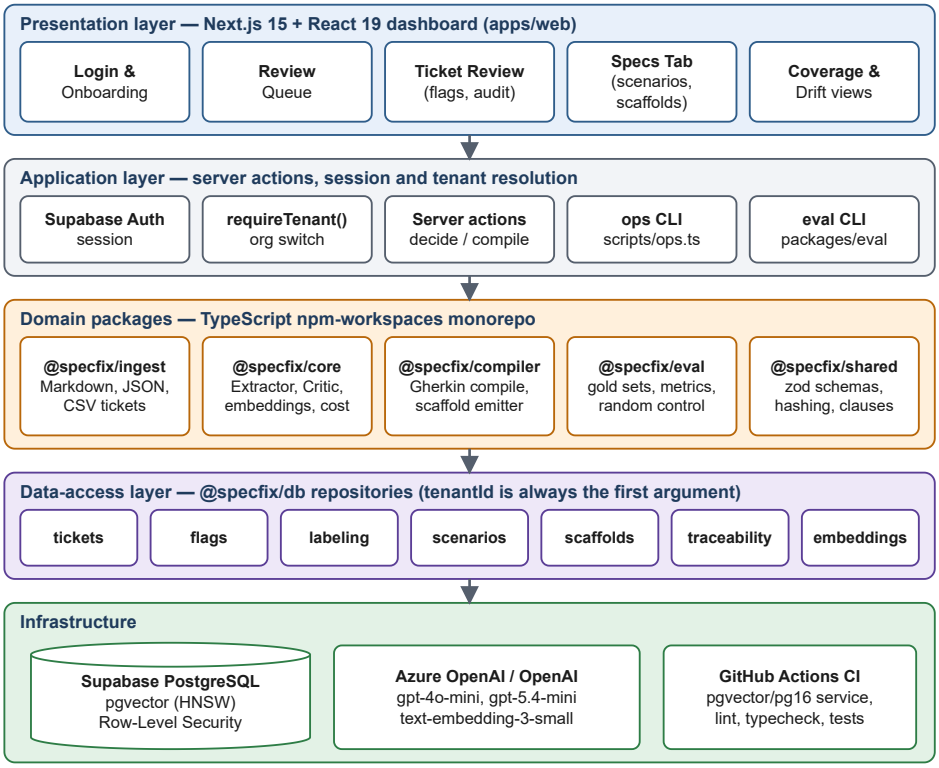


Figure 1.1: System Architecture of SpecFix

1.2 Objectives

Completed Objectives (60%)

1. Ingest and version tickets, and detect ambiguity with an LLM Extractor returning typed flags in six categories.
2. Filter false alarms with an Adversarial Critic whose safety rules are enforced in code; resist prompt injection.
3. Provide a review dashboard to accept, edit, dismiss or reopen flags, with an audit trail and a "mark ready" step.
4. Measure quality with blind-first labeling, precision/recall with confidence intervals and a random-pruning control.
5. Generate Gherkin scenarios and Playwright scaffolds behind human gates, with pgvector traceability and drift detection.
6. Isolate tenants and cap model spending per tenant; establish a generation evaluation baseline.

Pending Objectives (40%)

- Jira Cloud integration with a single, PM-triggered write-back at sign-off.

- A background worker (BullMQ on Redis) for analysis, compilation and embedding jobs.
- Independent blind labeling of real tickets, an optional conversational agent, and pilot deployment.

1.3 Summary of Similar Applications

Table 1.1: Comparison with Existing Tools and Approaches

Tool	Key Features	Gap Detection	Test Generation	Limitations
QVscribe / IBM RQA	NLP scoring of requirement statements	Yes	No	Built for regulated systems engineering; not ticket-native
Atlassian Rovo	AI in Jira, checks code against acceptance criteria	Partial	No	Works after code exists; no measured pre-development review
Qase / TestCollab AI	AI test-case generation	No	Yes	Generates tests from ambiguous requirements as written
Generic chat assistant	Ad-hoc “review this ticket”	Unstructured	Unstructured	No schema, audit trail or measurement; injectable
SpecFix (ours)	Extractor + Critic flags, human gate, Gherkin + Playwright scaffolds, traceability	Yes (measured)	Yes (gated)	Jira integration and worker pending

1.4 Organization of the Project Report

Chapter 2 covers requirements, Chapter 3 the design, Chapter 4 the implementation and interface, Chapter 5 testing, and Chapters 6–7 the conclusion and summary.

CHAPTER 2
SOFTWARE REQUIREMENT ANALYSIS

2.1 Introduction to Requirement Analysis

Requirements were derived from the project's Phase 1 and Phase 2 plans and from realistic, under-specified sample tickets (e.g. PAY-142 "Allow partial refunds"). The roles are the **Product Manager / Reviewer**, the **Organisation Admin**, the **Test Engineer** and the **Evaluator**, with the LLM service as an external system. Because accuracy judged by the prompt's own author only measures self-agreement, blind-first labeling and a control became requirements.

2.2 Feasibility Study

Table 2.1: Feasibility Study Summary

Type	Analysis	Conclusion
Technical	Models with strict JSON-schema output, PostgreSQL with pgvector on Supabase, Next.js, zod and Vitest are mature and free; all were prototyped and work in the repository.	Feasible
Operational	The dashboard mirrors how a PM already reads a ticket; one-click decisions add little overhead; generated tests are only scaffolds.	Highly Feasible
Economic	Open-source stack; measured model cost about US$0.0003 per ticket for analysis and US$0.0004 for scenario generation; per-tenant monthly token cap.	Cost-Effective
Legal / Security	Supabase Auth, Row-Level Security on every table, tenant-scoped repositories and an audit trail of decisions.	Feasible
Schedule	Phase 1 and Phase 2 milestones A–E are complete; Jira, worker and agent fit the remaining phase.	Feasible

2.3 Functional Requirements

Table 2.2: Functional Requirements of SpecFix

ID	Requirement	Description	Status
FR1	**Authentication & workspaces**	Sign up / sign in with Supabase Auth; create a tenant and project; roles owner, admin, reviewer; switch tenants.	Done
FR2	**Ticket ingestion**	Import Markdown, JSON and CSV tickets; new version only when the analysable text changes.	Done
FR3	**Ambiguity analysis**	Extractor produces flags: category, quoted span, what is unclear, why it matters, PM question, severity.	Done
FR4	**Adversarial Critic**	Optional second pass keeps or prunes each flag; pruned flags stored but hidden.	Done
FR5	**Flag review & ready**	Accept, edit, dismiss or reopen a flag with an audit trail; mark ready only when no flag is open.	Done
FR6	**Blind-first labeling**	Reviewer writes and locks their own gap list before flags are revealed.	Done
FR7	**Scenario generation & review**	Generate Gherkin from approved criteria; approve, edit, dismiss or reopen each scenario.	Done
FR8	**Scaffold generation**	Deterministic Playwright scaffold with placeholder selectors; review and download.	Done
FR9	**Coverage & drift**	Traceability links and embeddings; clause coverage; stale scenarios matched to the new version.	Done
FR10	**Evaluation & ops CLIs**	Gold sets, precision/recall, random-pruning control, generation metrics; seed, invite, compile, metrics.	Done
FR11	**Jira & worker**	Jira source with single write-back at sign-off; BullMQ background worker.	Pending

2.4 Non-Functional Requirements

Table 2.3: Non-Functional Requirements of SpecFix

Quality	Requirement and how it is met
Tenant isolation	`tenant_id` on every table, RLS enabled, every repository takes `tenantId` first, cross-tenant tests.
Output integrity	No LLM output used without zod validation; quoted spans verified against the ticket.

Quality	Requirement and how it is met
Injection resistance	Ticket text only inside neutralised delimiters in a user-role message; adversarial test fixtures.
Auditability	Append-only decision tables; every model call logged with prompt version, input, output, latency and cost.
Cost & reproducibility	Monthly token cap per tenant; prompts versioned by content hash; temperature 0 and fixed seed for evaluations.

2.5 Use Case Descriptions

Table 2.4: Main Use Cases

ID	Use Case	Actor	Main Flow
UC-01	**Analyze ticket**	Org Admin, LLM	Load latest version → Extractor returns JSON flags → validate, dedupe, verify spans → (Critic keep/prune) → store run, flags and call log.
UC-02	**Review flags**	Product Manager	Open ticket → accept / edit / dismiss each flag → decision appended → mark ready when no flag is open.
UC-03	**Generate specs**	PM, Test Engineer	Generate scenarios → approve / edit / dismiss → emit scaffold → fill selectors → accept and download.
UC-04	**Blind labeling & eval**	Reviewer, Evaluator	Write gap list → lock → flags revealed → link gaps to flags → run eval for precision, recall and control.

2.6 Module Descriptions

Table 2.5: Modules of SpecFix

Module	Responsibility
shared / ingest	zod schemas, flag taxonomy, hashing and clause splitting; `FileSource` for Markdown, JSON and CSV tickets.
core / compiler	Extractor, Critic, embeddings and cost accounting; Gherkin generation and the deterministic scaffold emitter.
db / eval	Seven migrations and tenant-scoped repositories; gold sets, precision/recall, control and generation metrics.
apps/web, scripts/ops.ts	Next.js dashboard (queue, ticket review, Specs tab) and operator CLI (provision, seed, invite, compile, metrics).

CHAPTER 3
SOFTWARE DESIGN

3.1 System Architecture Overview

SpecFix follows a layered architecture (Figure 1.1). The **presentation layer** is a Next.js 15 application using React Server Components and server actions, so no public REST API is exposed. The **application layer** resolves the user and tenant on every request. The **domain layer** is a set of TypeScript packages that return data only, and the **data-access layer** is the only code allowed to issue SQL. The **infrastructure** is Supabase (PostgreSQL with pgvector, Row-Level Security and authentication), Azure OpenAI / OpenAI endpoints and GitHub Actions CI. Two decisions run through the design: work belongs to a *ticket version* (identified by a content hash), so artefacts become *stale* instead of silently attaching to changed text; and tenant isolation is enforced in the repositories, by RLS and by a dedicated test suite.

3.2 Data Flow Diagrams

3.2.1 Level 0 DFD – Context Diagram

The context diagram (Figure 3.1) shows SpecFix as one process exchanging data with the ticket source, the LLM service, the product manager, the test engineer and the evaluator.

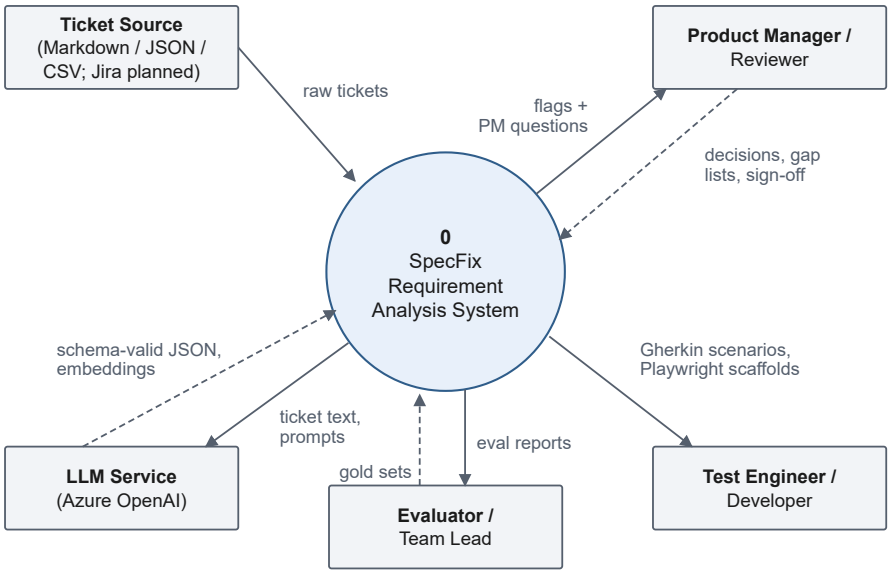


Figure 3.1: Level 0 DFD – Context Diagram

3.2.2 Level 1 DFD – Main Data Flow

The Level 1 DFD (Figure 3.2) decomposes the system into six processes — ingest and version, analyse, human review, compile specifications, traceability and embeddings, and evaluation — and six data stores.

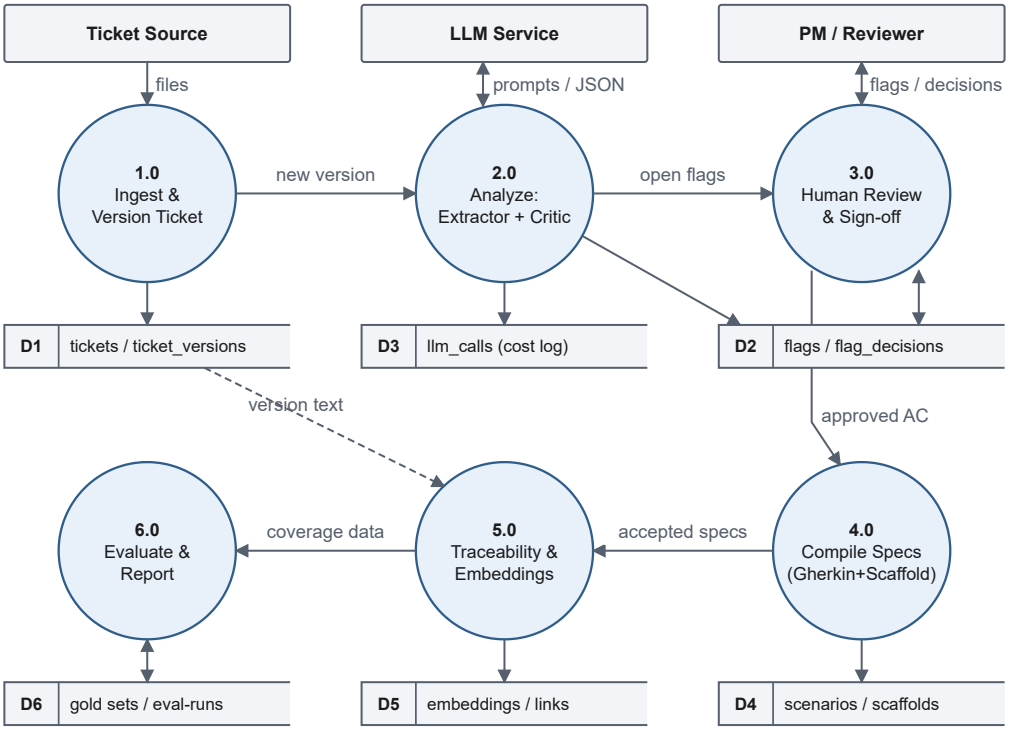


Figure 3.2: Level 1 DFD – Main Data Flow

3.3 UML Diagrams

3.3.1 Use Case Diagram

Four actors use the system (Figure 3.3). The Organisation Admin generalises the Product Manager and additionally manages workspaces, ingestion and analysis; the Test Engineer works on scenarios and scaffolds; the Evaluator runs evaluations and reads metrics.

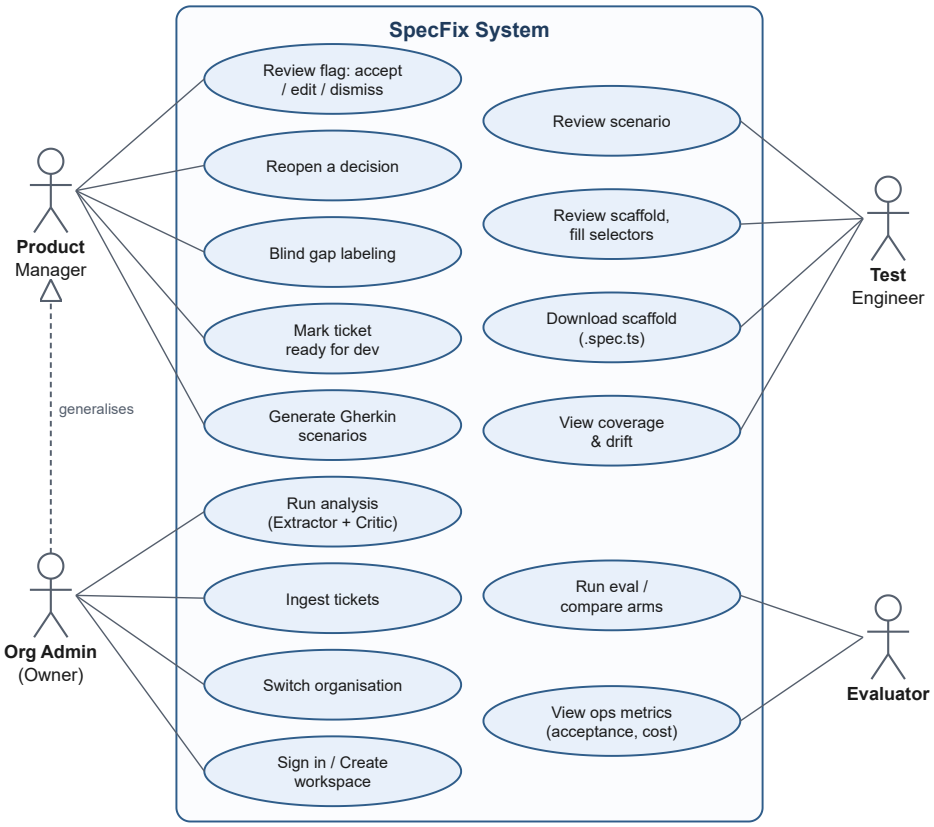


Figure 3.3: Use Case Diagram of SpecFix

3.3.2 Class Diagram

A Tenant owns Projects and Memberships; a Project owns Tickets; each Ticket has TicketVersions, each version has AnalysisRuns, and each run produces Flags with an append-only list of FlagDecisions (Figure 3.4). Scenarios are generated from resolved flags and can have one TestScaffold, an Embedding and traceability links.

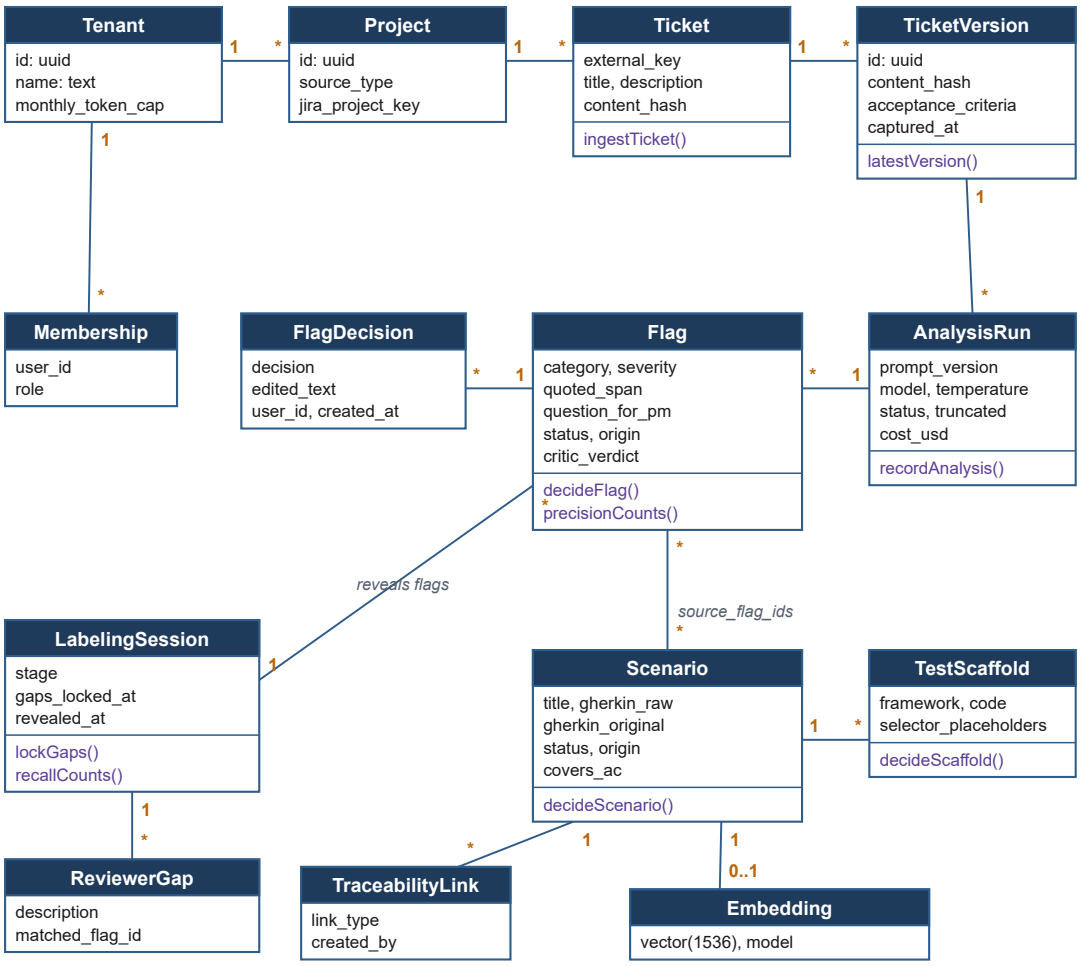


Figure 3.4: Class Diagram of the Core Domain

3.3.3 Sequence Diagram – Ticket Analysis

Figure 3.5 traces one analysis: the latest version is loaded through a tenant-scoped repository, the ticket is rendered with neutralised delimiters, the Extractor returns candidate flags, the output is validated and spans verified, the Critic returns keep/prune verdicts, safety rules are applied and the run, flags and every model call are stored.

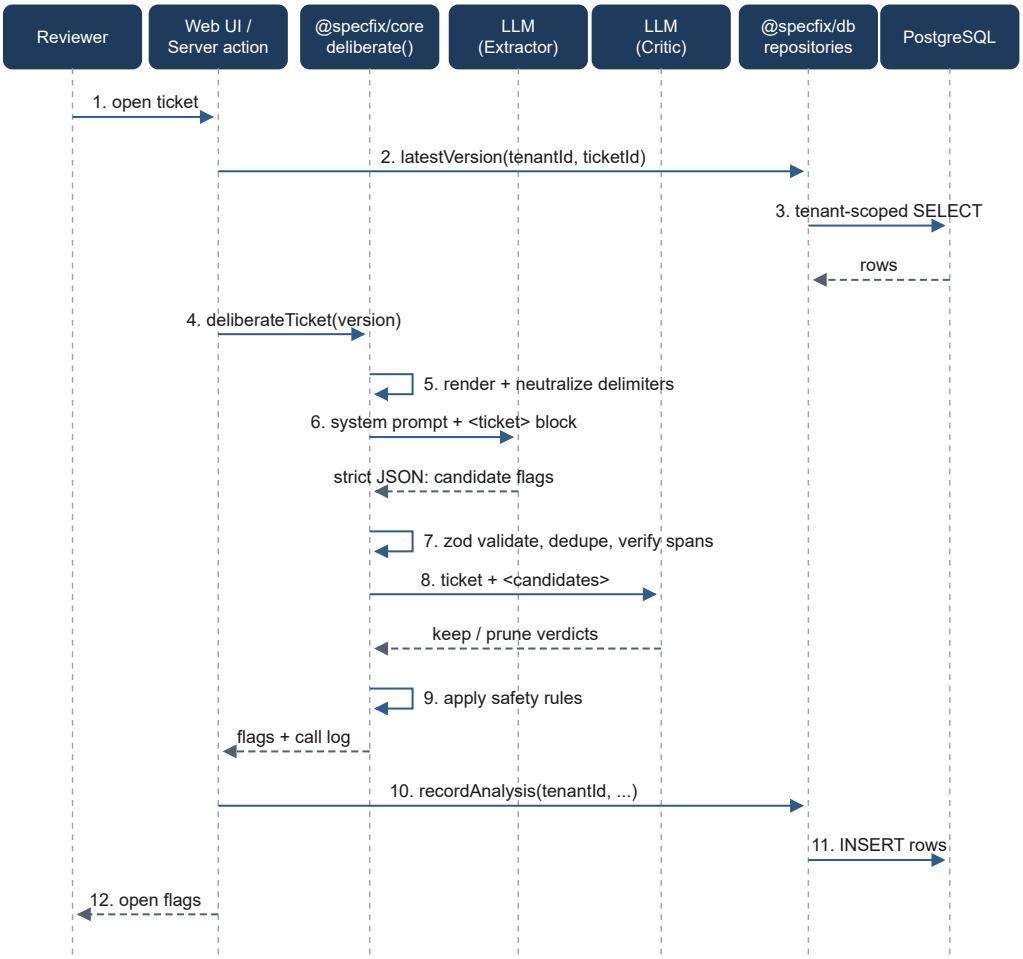


Figure 3.5: Sequence Diagram – Extractor and Critic Analysis

3.3.4 Entity Relationship (ER) Diagram

Figure 3.6 shows the fifteen principal tables and their one-to-many relationships; every tenant-scoped table also carries `tenant_id` with Row-Level Security. Scenario reviews, traceability links and three embedding tables complete the schema.

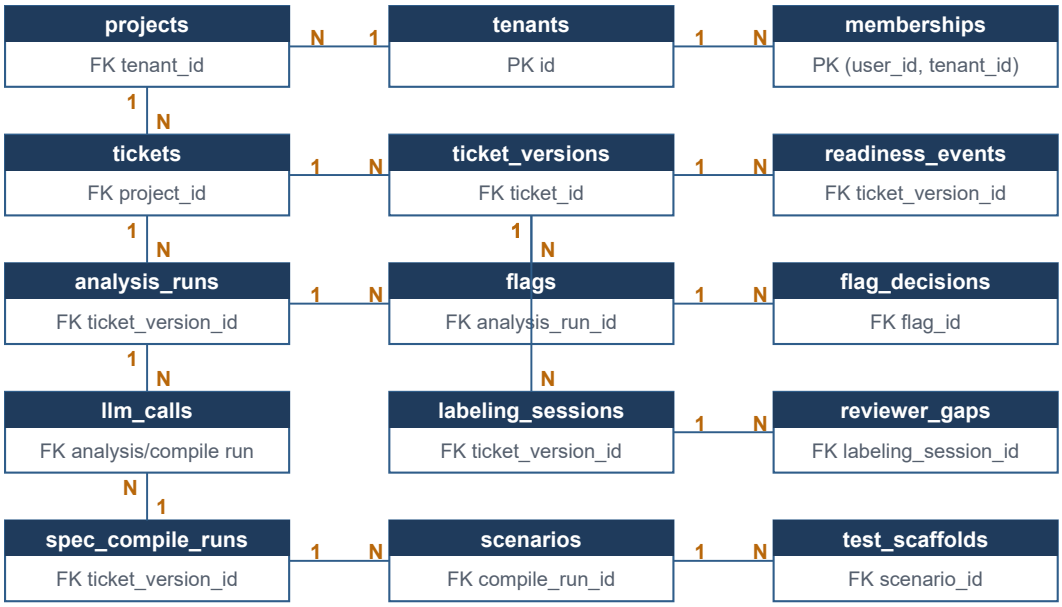


Figure 3.6: Entity Relationship Diagram

3.4 Database Design

PostgreSQL on Supabase is the only database; seven transactional migrations create the schema, and `llm_calls` logs every model call with its cost.

Table 3.1: Principal Database Tables

Table	Key Fields	Purpose
tenants / memberships	id, name, monthly_token_cap; (user_id, tenant_id) PK, role	Organisations, members and roles
tickets / ticket_versions	external_key, title, content_hash; frozen acceptance_criteria_text	Tickets and the versions all work is tied to
analysis_runs / flags	category, quoted_span, question_for_pm, severity, status, origin, critic_verdict	Detected gaps and their lifecycle
flag_decisions	flag_id FK, decision, edited_text, user_id	Append-only audit trail
scenarios / test_scaffolds / traceability_links	gherkin_raw, status, covers_ac; code, selector_placeholders; link_type	Generated specs and the traceability graph
***_embeddings**	content, embedding vector(1536), model; HNSW cosine index	Requirement, clause and scenario vectors

CHAPTER 4
IMPLEMENTATION AND USER INTERFACE

4.1 Technology Stack Used

Table 4.1: Technology Stack Summary

Layer	Technology	Reason for Choice
Language	TypeScript 5 (strict), Node.js ≥ 22, npm workspaces	One typed language across UI, server, CLI and tests
Frontend	Next.js 15 App Router, React 19, Tailwind CSS	Server components and server actions keep data access on the server
Auth & DB	Supabase Auth, PostgreSQL, pgvector, Row-Level Security	Relational integrity and vector search in one database
LLM	Azure OpenAI / OpenAI: gpt-4o-mini, gpt-5.4-mini, text-embedding-3-small	Strict JSON-schema output at a fraction of a cent per ticket
Validation	zod 3	Single source of truth for model output and types
Quality	Vitest 4, ESLint 9, Prettier 3, GitHub Actions (pgvector/pg16)	Tests, linting and migrations checked on every pull request

4.2 AI Architecture

Language models are used in three single-call steps — the Extractor, the Critic and the Spec Compiler — plus an embedding model. None is an autonomous agent: each call has a fixed schema, no tools and no memory, and its output is validated and then reviewed by a person. Prompts are versioned Markdown files whose version includes a hash of their content. Ticket text is sent only inside `<ticket>` delimiters in a user-role message, and delimiters inside the ticket are neutralised, so a ticket cannot close its data block and append instructions.

4.2.1 Extractor and Adversarial Critic

The Extractor returns up to 30 flags in six categories: missing information, vague language, contradiction, unhandled edge case, security/compliance gap and untestable criterion. Its output is parsed with a strict zod schema, duplicates are removed and every quoted span is checked to occur verbatim in the ticket. In the deliberated arm the Critic then returns keep/prune per candidate (Figure 4.1). Four rules are enforced in code: an unreviewed candidate is kept; the Critic cannot change category or quoted span; an empty refined question falls back to the original; and both passes use the same model unless the run is marked as differing.

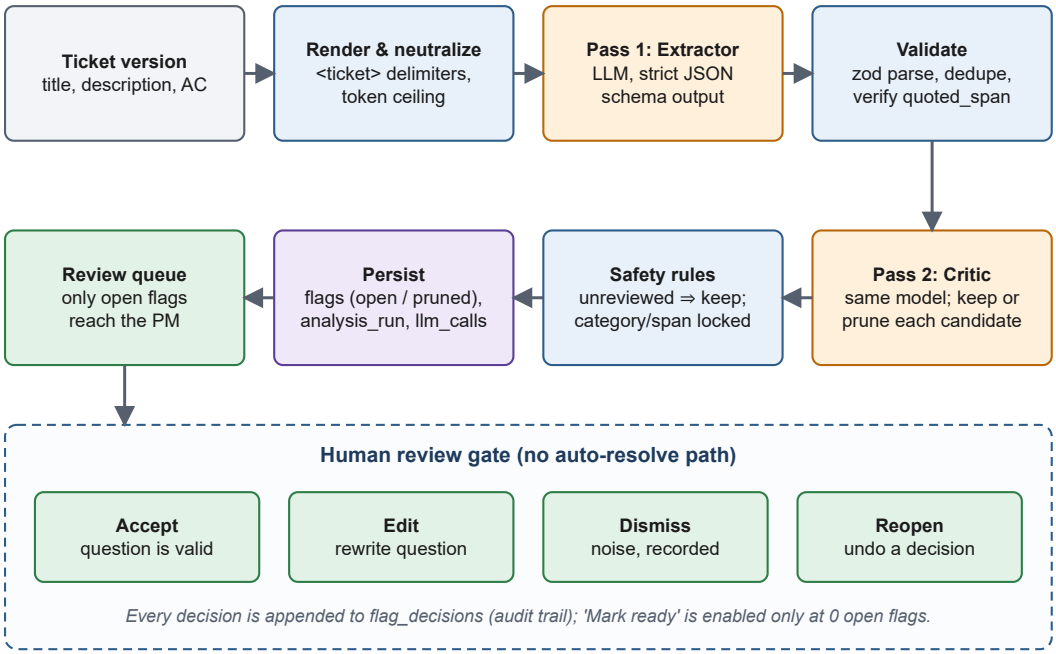


Figure 4.1: Two-Pass Analysis Pipeline and Human Review Gate

Random-pruning control. Pruning raises precision mechanically, even at random. The evaluation therefore compares the Critic with the same number of flags removed at random, 1000 times with a fixed seed. The Critic is kept only if $p < 0.05$ and no reviewer gap lost its only covering flag; the report builder refuses to produce a Critic-arm report without this control.

4.2.2 Spec Compiler, Scaffolds and Traceability

For an approved ticket version, the Spec Compiler writes one Gherkin scenario per testable behaviour, using the PM's answers as binding decisions and quoting the criterion each scenario covers, which is verified against the real text (Figure 4.2). The scaffold emitter is deliberately not an LLM call: it is a pure function of the scenario, emits `test.fixme` and a

named `PLACEHOLDER_*` locator for every interactive step, and lists them as a checklist. Every acceptance-criteria clause and accepted scenario is embedded (1536-dimension vectors with HNSW indexes); cosine similarity gives clause coverage and pairs stale scenarios with the closest clause of a changed ticket (drift).

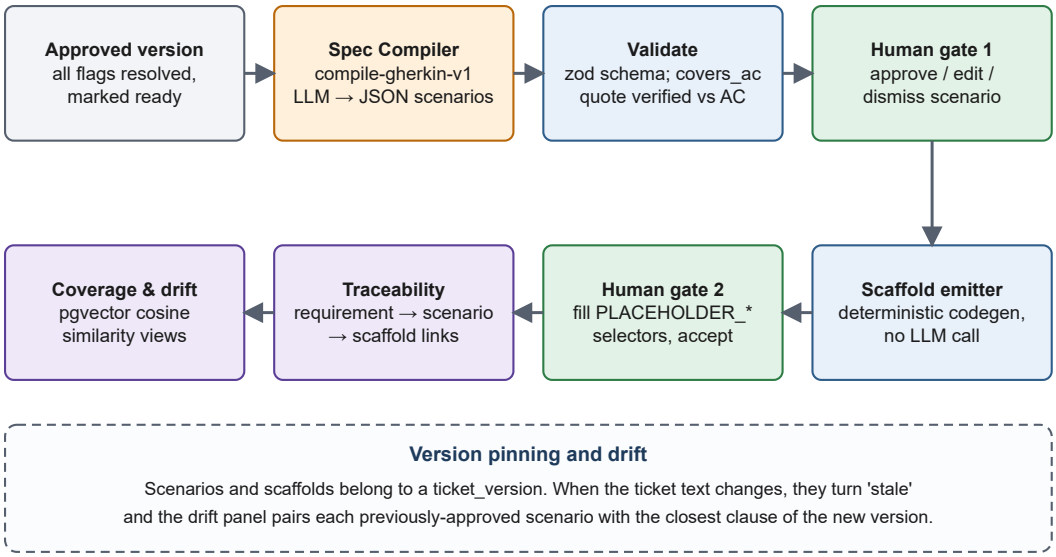


Figure 4.2: Spec Compiler Pipeline with Two Human Gates

Cost accounting. Every model call is logged with purpose, prompt version, request, response, latency, tokens and cost. On the 11-ticket gold set, analysis cost US$0.0030 in total (gpt-4o-mini) and scenario generation US$0.0044 (gpt-5.4-mini).

4.3 Frontend Implementation

Pages are React Server Components that call repositories on the server; each mutation is a server action that resolves the user and tenant first. Authentication uses Supabase cookies refreshed in middleware. The figures show the page layouts as implemented in `apps/web`, populated with the project's sample tickets.

4.3.1 Review Queue

The home page lists the project's tickets with a status badge — not analysed, open flags, flags resolved or ready for development (Figure 4.3). New users first create a workspace on the onboarding page.

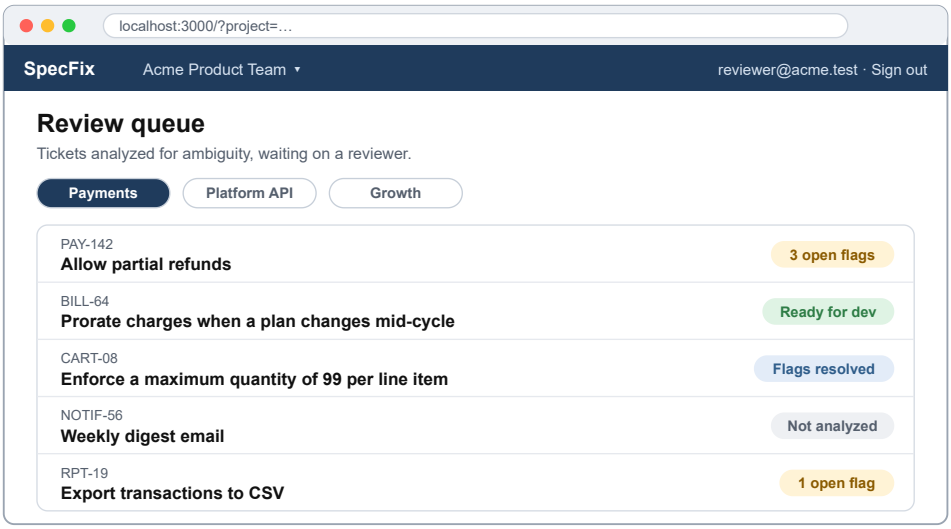


Figure 4.3: Review Queue Page

4.3.2 Ticket Review

The ticket page shows the ticket beside its flag cards with Accept, Edit, Dismiss and Reopen, followed by the decision history and readiness timeline (Figure 4.4). “Mark ready” stays disabled while any flag is open. For evaluation, the same page runs blind-first labeling: the reviewer’s own gap list is locked before flags are revealed, enforced by the server.

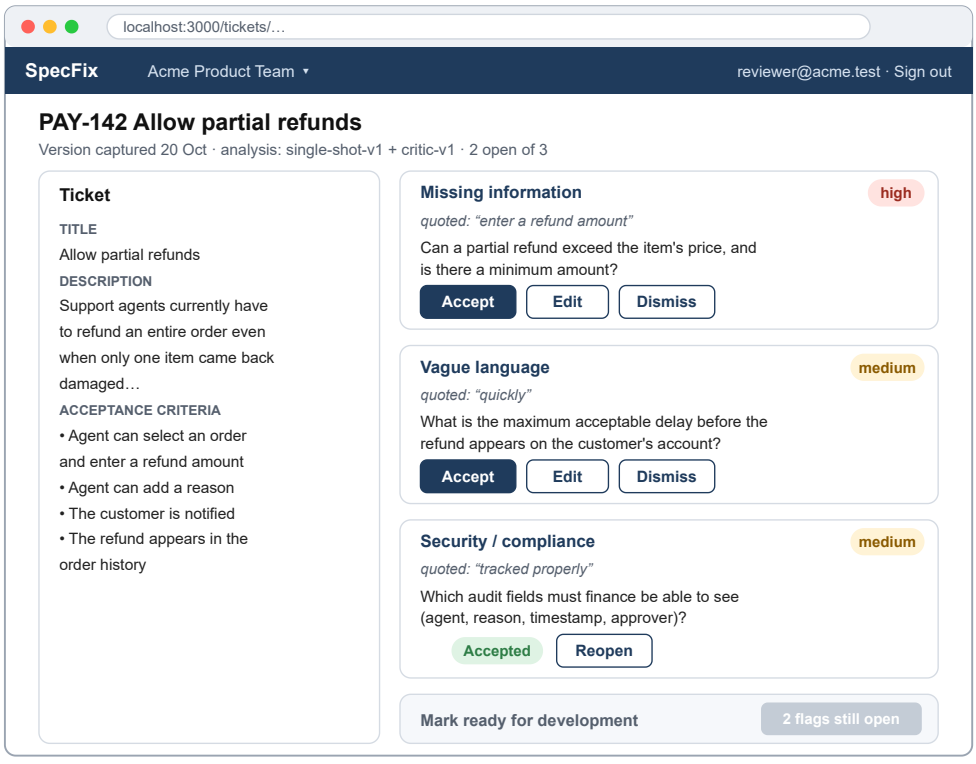


Figure 4.4: Ticket Review Page with Flag Cards

4.3.3 Specs Tab

The Specs tab generates scenarios, reviews each one, emits and reviews scaffolds with their placeholder checklist, and shows coverage and drift panels (Figure 4.5).

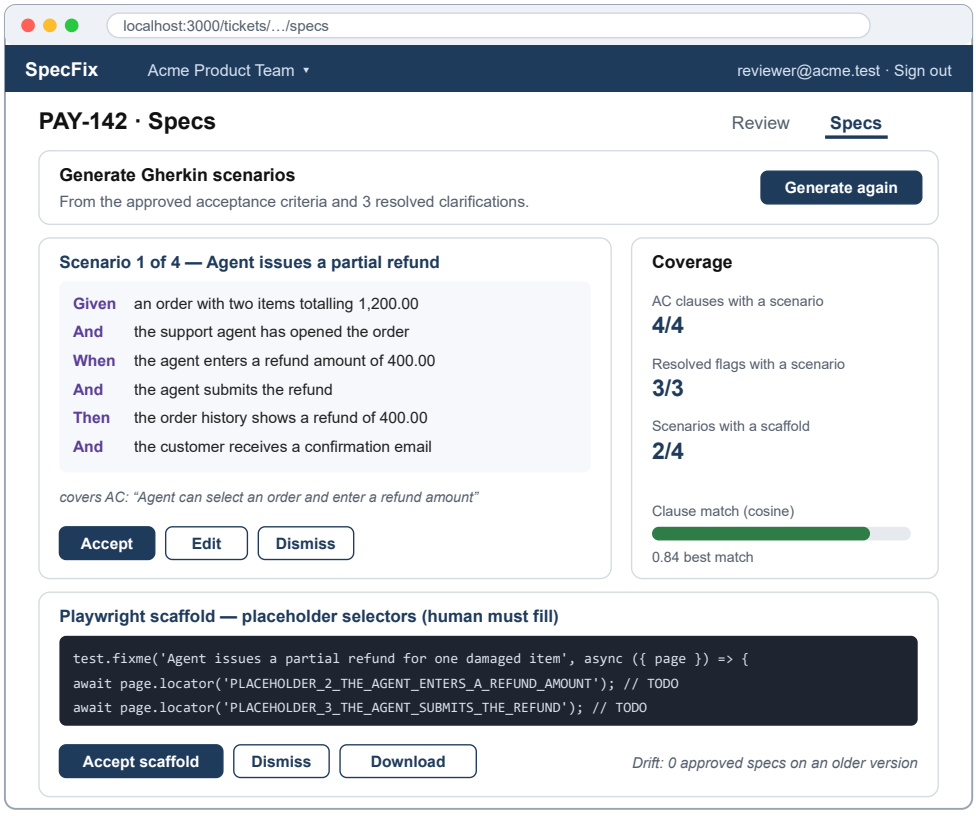


Figure 4.5: Specs Tab: Scenario Review, Coverage and Scaffold

CHAPTER 5
SOFTWARE TESTING

5.1 Testing Approach

Deterministic tests of everything around the model run on every change at no cost; evaluation runs call the real model on a versioned gold set and write committed reports. Database tests use only a disposable database.

5.2 Unit Testing

The Vitest suite has 320 tests in 22 files. Offline (model keys blanked) 270 pass and 50 are skipped (47 database-gated, 3 live-model); with a local PGlite database, 313 pass and 4 are skipped.

Table 5.1: Unit Testing Summary by Package (offline run)

Package	Passed	Skipped	What is covered
shared	26	0	Content-hash stability, clause splitting and cover matching
ingest	17	0	CSV quoting, CRLF, Markdown and JSON parsing
core	74	2	Schemas, rendering, Critic safety rules, embeddings, injection
compiler	15	1	Gherkin schema, cover verification, scaffold emission
eval	89	0	Precision/recall, Wilson intervals, control, gold sets, generation metrics
db	49	47	Embedding sync; tenant isolation, labeling order, scenario reopen (DB-gated)

5.3 Test Case Specifications

Table 5.2: Black Box and White Box Test Cases

ID	Test Case	Condition	Expected	Result
BB-01	Extra property in output	Flag with an unknown key	Rejected by strict schema	Pass

ID	Test Case	Condition	Expected	Result
BB-02	Unverified quoted span	Quote absent from ticket	Span cleared and counted	Pass
BB-03	Delimiter escape	Ticket contains `</ticket>` + instructions	Neutralised; block intact	Pass
BB-05	Reveal before lock	Gap list not locked	Refused by the server	Pass*
WB-02	Control, judging Critic	Prunes exactly the noise	$p < 0.05$	Pass
WB-03	Control, random Critic	Prunes at random	Does not beat chance	Pass

* Database-gated test; passed on the local PGlite run and in CI.

5.4 Multi-Tenant Isolation Testing

The isolation suite creates two tenants and calls every repository with tenant A's id and tenant B's object ids: B's data stays invisible, writes to B's flags, tickets, sessions and vectors are refused, and usage metrics count only A's rows. All checks pass in CI and on the local database run.

5.5 Evaluation Results

The generation baseline ran over the 11-ticket gold set with `compile-gherkin-v1` on gpt-5.4-mini (temperature 0), producing 42 scenarios. Flag-detection precision and recall are not quoted yet because no independent reviewer verdicts exist.

Table 5.3: Generation Evaluation Results (gold-v1)

Metric	Definition	Result	Wilson 95% CI
AC coverage	AC clauses with at least one verified covering scenario	44/48 = 91.7%	80.4% – 96.7%
Hallucinated cover	Scenarios whose claimed coverage is not in the criteria	1/42 = 2.4%	0.4% – 12.3%
Scaffold validity	Structurally valid Playwright scaffolds	42/42 = 100%	91.6% – 100%

CHAPTER 6
CONCLUSION

SpecFix shows that a language model can be a dependable part of a requirements workflow when the system around it is engineered carefully. The platform ingests and versions tickets, detects six kinds of ambiguity with an Extractor and Critic, resists prompt injection, lets a human resolve every flag with a full audit trail, and turns approved tickets into Gherkin scenarios and Playwright scaffolds behind further human gates. A pgvector traceability graph reports clause-level coverage and drift, on a multi-tenant schema whose isolation is tested. Every quality figure comes from a reproducible report with confidence intervals, and figures that have not been independently measured are reported as such.

6.1 Current Status & Future Work

Table 6.1: Project Status by Milestone

Milestone	Scope	Status
Phase 1 – flagging core	Ingestion, Extractor, Critic, injection defence, dashboard, labeling, eval harness, control	Complete
Phase 2 A–E	Spec Compiler, scenario review UI, scaffold emitter, traceability and embeddings, generation eval	Complete
Independent validation	Blind labels by non-authors, frozen gold set, Critic-arm report with control	Pending
Jira integration & worker	Jira source, single write-back at sign-off, BullMQ worker	Pending
Phase 2 F & deployment	Optional conversational agent (A/B tested), pilot deployment	Pending

The next priority is independent validation on 20–30 real tickets labeled blind by reviewers who did not write the prompts; if the Critic does not beat its control it will be removed rather than retuned. After that come Jira integration, the background worker, and — only if justified by cost and A/B results — the conversational agent.

CHAPTER 7
SUMMARY

7.1 Summary

SpecFix is an AI-assisted requirements-quality platform for ordinary software teams. It finds the places in a ticket where a developer would have to guess and turns each into a precise question for the product manager; a second model pass removes false alarms, a human resolves the rest, and the ticket is marked ready with a full record of what was decided. Approved criteria then become Gherkin scenarios and Playwright scaffolds, linked by a traceability graph that shows coverage and drift.

The system is a TypeScript monorepo of seven packages with seven SQL migrations, a Next.js dashboard, an operator CLI and CI. Its defining traits are safety by construction (validated output, ticket text as data), humans in control (no automatic resolution), honest measurement (versioned prompts and gold sets, confidence intervals, a random-pruning control) and tenant isolation with per-tenant cost caps. With 270 offline tests passing (313 with the database suites) and a generation baseline of 91.7% AC coverage at about US$0.0004 per ticket, the project is ready for independent validation and Jira integration.

REFERENCES

1. ISO/IEC/IEEE 29148:2018, *Systems and software engineering – Life cycle processes – Requirements engineering*. ISO, 2018.
2. B. W. Boehm, *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice-Hall, 1981.
3. A. Ferrari, F. Dell'Orletta, A. Esuli, V. Gervasi and S. Gnesi, "Natural Language Requirements Processing: A 4D Vision," *IEEE Software*, vol. 34, no. 6, pp. 28–35, 2017.
4. A. Mavin, P. Wilkinson, A. Harwood and M. Novak, "Easy Approach to Requirements Syntax (EARS)," in *Proc. 17th IEEE Int. Requirements Engineering Conf.*, 2009, pp. 317–322.
5. E. B. Wilson, "Probable Inference, the Law of Succession, and Statistical Inference," *J. American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.
6. P. Good, *Permutation Tests: A Practical Guide to Resampling Methods for Testing Hypotheses*, 2nd ed. Springer, 2000.
7. K. Greshake et al., "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in *Proc. 16th ACM Workshop on AI and Security*, 2023.
8. Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE TPAMI*, vol. 42, no. 4, pp. 824–836, 2020.
9. M. Wynne and A. Hellesøy, *The Cucumber Book: Behaviour-Driven Development for Testers and Developers*. Pragmatic Bookshelf, 2012.
10. OpenAI, "Structured Outputs" and "Embeddings" guides. [Online]. Available: https://platform.openai.com/docs
11. Supabase, "Row Level Security" documentation. [Online]. Available: https://supabase.com/docs
12. pgvector, "Open-source vector similarity search for Postgres." [Online]. Available: https://github.com/pgvector/pgvector
13. Vercel, "Next.js Documentation." [Online]. Available: https://nextjs.org/docs
14. Microsoft, "Playwright Test Documentation." [Online]. Available: https://playwright.dev/docs/intro

APPENDIX A: ABBREVIATIONS

Term	Full Form	Term	Full Form
AC	Acceptance Criteria	**HNSW**	Hierarchical Navigable Small World
AI	Artificial Intelligence	**JSON**	JavaScript Object Notation
API	Application Programming Interface	**LLM**	Large Language Model
BDD	Behaviour-Driven Development	**PM**	Product Manager
CI	Continuous Integration	**RLS**	Row-Level Security
CLI	Command-Line Interface	**SQL**	Structured Query Language
DFD	Data Flow Diagram	**UI**	User Interface
ER	Entity Relationship	**UML**	Unified Modeling Language

APPENDIX B: SAMPLE COMMANDS

Command	Purpose
`npm run ops -- seed --owner <email> --tenant "Acme" --analyze`	Create workspace, ingest and analyse fixture tickets
`npm run ops -- invite --owner <email> --tenant "Acme" --email <email>`	Add an independent reviewer
`npm run ops -- compile --ticket fixtures/tickets/sample/PAY-142.md`	Compile one ticket to Gherkin
`npm run eval -- run --set gold-v1 --prompt single-shot-v1 --deliberate`	Run the Critic arm with its control
`npm run eval -- generate --set gold-v1 --prompt compile-gherkin-v1`	Run the generation evaluation
`npm run verify`	Format check, lint, typecheck and tests